

Core Java Notes

Core Java — Complete Notes

1. Introduction to Java

Java is a **high-level, object-oriented, platform-independent** programming language developed by **Sun Microsystems** in 1995, now owned by **Oracle**. It was designed with the philosophy of “Write Once, Run Anywhere” — meaning a program written on one machine can run on any other machine that has a Java Virtual Machine, without needing to be rewritten. This is very different from languages like C/C++, where code compiled on one operating system usually cannot run directly on another.

Java achieves this by not compiling source code directly into machine code. Instead, it compiles code into an intermediate form called **bytecode**, which is then interpreted/executed by the JVM on whichever machine it runs on. This one extra layer is what gives Java its portability.

Key Features (explained):

- **Platform Independent** — “Write Once, Run Anywhere” (WORA). The `.class` bytecode file is the same no matter which OS compiled it; only the JVM differs per platform.
- **Object-Oriented** — Java organizes code around objects (real-world entities) that bundle data and behavior together, making large programs easier to design, maintain, and reuse.
- **Simple & Secure** — No explicit pointers (so no direct memory address manipulation), automatic garbage collection (you don’t manually free memory), and a robust security model via the classloader and bytecode verifier.
- **Robust** — Strong compile-time type checking, runtime checking, and a mandatory exception-handling mechanism reduce the chances of a program crashing unexpectedly.
- **Multithreaded** — Built-in language and library support lets a Java program perform multiple tasks (threads) at the same time, improving performance on multi-core systems.
- **Architecture Neutral** — The compiled bytecode does not depend on any specific CPU architecture; only the JVM implementation for that architecture needs to exist.

JVM, JRE, JDK

These three terms are frequently confused, but each plays a distinct role in the Java ecosystem:

Term	Full Form	Purpose
JVM	Java Virtual Machine	The engine that actually executes bytecode (<code>.class</code> files). It is platform-dependent (a different JVM build exists for Windows, Linux, macOS) but makes the bytecode running on top of it platform-independent. It loads code, verifies it, and interprets/JIT-compiles it into native machine instructions.
JRE	Java Runtime Environment	JVM + the standard class libraries (like <code>java.lang</code> , <code>java.util</code>) needed to actually run a compiled Java program. If you only want to run Java applications (not write your own), the JRE is enough.
JDK	Java Development Kit	JRE + development tools such as the compiler (<code>javac</code>), debugger, and documentation generator. This is what a developer installs in order to write and compile Java programs.

In short: **JDK** $\supset$ **JRE** $\supset$ **JVM** — the JDK contains the JRE, and the JRE contains the JVM.

Compilation and execution flow:

```
.java file → javac (compiler) → .class file (bytecode) → JVM → machine code
```

You write human-readable code in a `.java` file. The `javac` compiler translates it into bytecode stored in a `.class` file. When you run the program, the JVM loads this bytecode, verifies it for safety, and either interprets it line by line or uses the **JIT (Just-In-Time) compiler** to convert frequently used parts into native machine code for faster execution.

2. Basic Structure of a Java Program

Every Java application needs at least one class and a `main` method, which acts as the entry point — the very first piece of code the JVM looks for and runs when you start the program.

```
public class HelloWorld {
    public static void main(String[] args) {
```

```

        System.out.println("Hello, World!");
    }
}

```

Line-by-line explanation: - `public class HelloWorld` — declares a class named `HelloWorld`. In Java, the file name **must** match the public class name exactly (so this file must be saved as `HelloWorld.java`), otherwise the compiler throws an error. - `public static void main(String[] args)` — this exact signature is what the JVM searches for to start execution: - `public` — the method must be accessible from outside the class, since the JVM (an external process) needs to call it. - `static` — allows the JVM to call `main` without first creating an object of the class (because at startup, no objects exist yet). - `void` — the method does not return any value back to the JVM. - `String[] args` — an array that receives command-line arguments passed when running the program, e.g. `java HelloWorld arg1 arg2`. - `System.out.println("Hello, World!");` — `System.out` is a pre-built `PrintStream` object representing the standard output (console); `println()` prints the given text followed by a new line.

3. Data Types

Java is a **strongly, statically typed** language — every variable must be declared with a specific type before use, and that type cannot change. Data types are split into two broad categories.

Primitive Data Types (8 total)

Primitives store actual values directly in memory (not references), which makes them fast and memory-efficient. They are built into the language itself, not defined via any class.

Type	Size	Default	Range/Use
<code>byte</code>	1 byte	0	-128 to 127 — used to save memory in large arrays
<code>short</code>	2 bytes	0	-32,768 to 32,767
<code>int</code>	4 bytes	0	-2^{31} to $2^{31}-1$ — the default choice for whole numbers
<code>long</code>	8 bytes	0L	-2^{63} to $2^{63}-1$ — used when <code>int</code> range isn't enough
<code>float</code>	4 bytes	0.0f	single-precision decimal, less precise, saves memory
<code>double</code>	8 bytes	0.0d	double-precision decimal — the default for decimal numbers
<code>char</code>	2 bytes	'000'	stores a single 16-bit Unicode character
<code>boolean</code>	1 bit (JVM-dependent)	false	stores only <code>true</code> or <code>false</code> , used for logical conditions

Non-Primitive (Reference) Data Types

Unlike primitives, reference types (Strings, Arrays, Classes, Interfaces, etc.) do not store the actual data in the variable itself. Instead, the variable stores a **reference (memory address)** pointing to where the actual object lives on the heap. This is why comparing objects with `==` compares references, not content (explained further in the String section).

4. Variables

A variable is a named container that holds a value of a specific data type. Java has three kinds of variables based on where they are declared and how long they live.

```
int age = 25;           // local variable
```

Types of variables (explained): 1. **Local variable** — declared inside a method, constructor, or block. It only exists while that method is executing, and must be initialized before use (Java gives no default value to local variables). 2. **Instance variable** — declared inside a class but outside any method. Each object (instance) of the class gets its own separate copy. These are initialized to default values (0, null, false, etc.) automatically if not explicitly set. 3. **Static variable** — declared with the `static` keyword. Only **one copy** exists for the entire class, shared by all objects — changing it through one object affects the value seen by every other object.

```

class Demo {
    int instanceVar = 10;           // instance variable — one copy per object
    static int staticVar = 20;      // static variable — one copy shared by all objects
}

```

```

void method() {
    int localVar = 30;    // local variable – exists only inside this method
}

```

5. Operators

Operators are special symbols that perform operations on variables and values.

Category	Operators	Purpose
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	basic math operations; <code>%</code> gives the remainder
Relational	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	compares two values, results in <code>true</code> / <code>false</code>
Logical	<code>&&</code> <code>& \ </code> <code>!</code>	combines boolean expressions (AND, OR, NOT)
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	assigns or updates a variable's value
Unary	<code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>!</code>	operates on a single operand (increment, decrement, negation)
Bitwise	<code>&</code> <code>\ </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> <code>>>></code>	operates directly on individual bits of a number
Ternary	<code>condition ? value1 : value2</code>	shorthand for a simple if-else that returns a value
instanceof	<code>—</code>	checks whether an object is an instance of a given class/interface

Example:

```

int a = 10, b = 3;
System.out.println(a % b);    // 1 → remainder of 10/3
System.out.println(a > b ? "A" : "B");    // A → since a > b is true, "A" is chosen

```

6. Control Statements

Control statements decide the order in which individual instructions/blocks of a program are executed, allowing decisions and repetition.

Conditional Statements

`if-else` chooses between different blocks of code based on a boolean condition. `switch` is useful when comparing one variable against many possible fixed values, often more readable than a long `if-else` chain.

```

if (condition) {
    // runs if condition is true
} else if (condition) {
    // runs if the first condition is false but this one is true
} else {
    // runs if none of the above are true
}

switch (variable) {
    case value1:
        // code for value1
        break;    // exits the switch; without it, execution "falls through" to the next case
    default:
        // runs if no case matches
}

```

Loops

Loops repeat a block of code as long as a given condition holds true, avoiding repetitive manual code.

```
for (int i = 0; i < 5; i++) { }           // runs a known number of times (initialization; condition; update)

while (condition) { }                   // repeats as long as condition is true, checked BEFORE each run

do {
} while (condition);                     // like while, but checked AFTER each run, so it executes at least once

for (int x : array) { }                 // enhanced for-each loop – iterates directly over array/collection elements
```

Jump statements: - `break` — immediately exits the nearest enclosing loop or switch block. - `continue` — skips the rest of the current iteration and moves to the loop's next iteration. - `return` — exits the current method entirely, optionally sending a value back to the caller.

7. Arrays

An array is a fixed-size, ordered collection of elements of the **same data type**, stored in contiguous memory locations. Arrays let you group related values under a single variable name instead of declaring separate variables for each.

```
int[] arr = new int[5];                 // declares an array and allocates memory for 5 ints (all default to 0)
int[] arr2 = {1, 2, 3, 4, 5};           // declares and immediately initializes with given values

int[][] matrix = new int[3][3];         // a 2D array – essentially an array of arrays (3 rows, 3 columns)

System.out.println(arr.length);         // .length gives the number of elements (note: no parentheses, it's a field, not a method)
```

Key points (explained): - Arrays are **objects** in Java, so they are stored on the heap, and array variables actually hold a reference to that memory, similar to other reference types. - Indexing starts from **0**, so the last valid index of an array of size **n** is **n-1**. Accessing index **n** or beyond throws `ArrayIndexOutOfBoundsException`. - Once created, an array's size is **fixed** — it cannot grow or shrink. (For a resizable structure, use `ArrayList` from the Collections Framework instead.)

8. Object-Oriented Programming (OOP) Concepts

OOP is a programming paradigm built around the idea of “objects” — self-contained units that combine data (fields) and behavior (methods). Java is fundamentally an object-oriented language, and its design rests on four core pillars: **Encapsulation, Inheritance, Polymorphism, and Abstraction**. Together, these principles make code more modular, reusable, and easier to maintain, especially as applications grow large.

8.1 Class and Object

- **Class** — a blueprint or template that defines what properties (fields/state) and behaviors (methods) objects of that type will have. A class itself does not occupy memory for object data; it's just a design.
- **Object** — an actual instance created from a class, using the `new` keyword. Each object gets its own memory and its own copy of instance variables (unless static).

```
class Car {
    String color;           // field – represents the state of a Car object
    void drive() {          // method – represents the behavior of a Car object
        System.out.println("Driving...");
    }
}

class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // 'new' creates an actual object in memory; myCar holds a reference to it
        myCar.color = "Red";   // setting the object's field
        myCar.drive();         // calling the object's method
    }
}
```

8.2 Constructor

A constructor is a special block of code, resembling a method, that is automatically called when an object is created using `new`. Its main purpose is to initialize the object's fields with starting values. A constructor has the **same name as the class** and has **no return type**, not even `void`.

```

class Car {
    String color;

    Car() { // default (no-argument) constructor
        color = "White";
    }

    Car(String c) { // parameterized constructor – accepts arguments to customize the object
        color = c;
    }
}

```

Important points: - Constructor overloading — a class can have multiple constructors as long as their parameter lists differ, giving flexibility in how objects are created. - **this()** — used inside one constructor to call another constructor of the same class, useful to avoid duplicating initialization code. - If you don't write any constructor yourself, Java automatically supplies a free **default no-argument constructor**. However, as soon as you define even one constructor, this automatic one disappears.

8.3 Encapsulation

Encapsulation means bundling data (fields) and the methods that operate on that data into a single unit (a class), while **restricting direct access** to the internal details from outside. This is typically done by marking fields as **private** and exposing controlled access through **public** getter and setter methods.

```

class Employee {
    private double salary; // hidden from outside classes – cannot be accessed directly as employee.salary

    public double getSalary() { return salary; } // controlled "read" access
    public void setSalary(double s) {
        if (s > 0) salary = s; // validation logic – prevents invalid data from ever being set
    }
}

```

Why it matters: Encapsulation protects an object's internal state from being changed in unexpected or invalid ways, since all access must go through methods that can enforce rules (like the salary validation above). It also means the internal implementation can change later without breaking code that uses the class, as long as the public methods stay the same.

8.4 Inheritance

Inheritance lets one class (the **subclass** or **child class**) automatically acquire the fields and methods of another class (the **superclass** or **parent class**), using the **extends** keyword. This promotes code reuse — common behavior is written once in the parent and reused by every child.

```

class Animal {
    void eat() { System.out.println("eating..."); }
}

class Dog extends Animal { // Dog inherits eat() from Animal, and adds its own bark()
    void bark() { System.out.println("barking..."); }
}

```

Types of Inheritance: - **Single** — one class inherits from one parent ($A \rightarrow B$). - **Multilevel** — a chain of inheritance ($A \rightarrow B \rightarrow C$), where C inherits from B, which itself inherits from A. - **Hierarchical** — multiple classes inherit from the same parent ($A \rightarrow B, A \rightarrow C$). - **Multiple inheritance through classes is NOT supported in Java** (a class cannot extend two classes at once). This is a deliberate design choice to avoid the ambiguity known as the "Diamond Problem" (if two parents had a method with the same signature, the compiler wouldn't know which to inherit). Java instead achieves similar flexibility through **interfaces**, which a class can implement multiple of.

super keyword — used inside a subclass to refer to its immediate parent class, commonly to call the parent's overridden method or its constructor:

```

class Dog extends Animal {
    void eat() {
        super.eat(); // explicitly calls Animal's eat() first
        System.out.println("dog eating"); // then adds Dog-specific behavior
    }
}

```

8.5 Polymorphism

Polymorphism literally means "many forms" — it refers to the ability of the same method name or the same reference type to behave differently depending on context. Java supports two kinds:

a) Compile-time Polymorphism (Method Overloading) — multiple methods share the same name but differ in the number or type of parameters. The compiler decides which version to call based on the arguments passed, at compile time itself:

```
void add(int a, int b) { }           // version 1
void add(int a, int b, int c) { }    // version 2 – different parameter count
void add(double a, double b) { }     // version 3 – different parameter type
```

b) Runtime Polymorphism (Method Overriding) — a subclass provides its own specific implementation of a method that is already defined in its parent class, with the exact same signature. Which version actually runs is decided at **runtime**, based on the real object type — this is called dynamic method dispatch:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); } // overrides parent's version
}

Animal a = new Cat(); // upcasting: a Cat object referenced as an Animal type
a.sound();             // prints "Meow" – Java looks at the ACTUAL object type at runtime, not the reference type
```

Rules for overriding: - Method name, parameter list, and return type (or a covariant/compatible subtype) must match exactly with the parent's method. - The overriding method cannot have a more restrictive access modifier than the one it overrides (e.g., cannot go from `public` to `private`). - The `@Override` annotation isn't mandatory, but is strongly recommended — it lets the compiler catch mistakes (like typos in the method name) that would otherwise silently create a brand-new method instead of overriding.

8.6 Abstraction

Abstraction means hiding complex implementation details and exposing only the essential, relevant features to the user of a class. It lets you focus on “what” an object does rather than “how” it does it. Java provides two mechanisms for abstraction:

a) Abstract class — a class that can contain both abstract methods (declared without a body, forcing subclasses to implement them) and regular concrete methods (with a body, shared by all subclasses):

```
abstract class Shape {
    abstract void draw(); // no body – every subclass MUST provide its own implementation
    void display() {      // normal, fully-implemented method – inherited as-is
        System.out.println("Shape");
    }
}
class Circle extends Shape {
    void draw() { System.out.println("Drawing Circle"); } // provides the mandatory implementation
}
```

- An abstract class **cannot be instantiated directly** (you can't write `new Shape()`), since it may be incomplete.
- It can still have constructors, instance fields, and static methods, unlike an interface (in older Java versions).

b) Interface — traditionally, a 100% abstract contract that only specifies **what** methods a class must implement, without saying how:

```
interface Drawable {
    void draw(); // implicitly public and abstract
}
class Square implements Drawable {
    public void draw() { System.out.println("Drawing Square"); } // must implement, or Square becomes abstract too
}
```

- All fields declared in an interface are implicitly `public static final` (constants).
- Unlike classes, a single class **can implement multiple interfaces**, which is how Java offers the benefits of multiple inheritance without its pitfalls.
- Since **Java 8**, interfaces can also have `default` methods (with a body, providing a fallback implementation) and `static` methods.
- Since **Java 9**, interfaces can also have `private` methods, used internally to share code between default methods.

9. Access Modifiers

Access modifiers control the **visibility/accessibility** of classes, fields, methods, and constructors from other parts of a program. Choosing the right modifier is a key part of designing a well-encapsulated class.

Modifier	Same Class	Same Package	Subclass (different package)	Everywhere (World)
<code>private</code>	Yes	No	No	No
default (no modifier)	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

- `private` — most restrictive; typically used for internal fields that should only be manipulated through the class's own methods (core to encapsulation).
- default (no keyword written) — accessible only within the same package.
- `protected` — accessible within the same package, and also by subclasses even if they're in a different package (useful for inheritance-based designs).
- `public` — least restrictive; accessible from anywhere the class is visible.

10. Important Keywords

`static`

The `static` keyword means a member belongs to the **class itself**, rather than to any individual object. This means it is loaded into memory once, when the class is first loaded by the JVM, and shared by every object of that class.

```
class Counter {
    static int count = 0;    // shared across all objects - incrementing it via one object affects all
    static void show() { }  // can be called directly using the class name, without creating an object
}
```

- Static variables and methods are commonly used for values/behavior that logically belong to the class as a whole (e.g., a counter of how many objects have been created).
- Static methods **cannot directly access non-static (instance) members**, because instance members belong to a specific object, and a static context has no particular object tied to it.
- **Static block** — a block of code that runs exactly once, automatically, when the class is loaded — commonly used for one-time static initialization:

```
static {
    System.out.println("Static block executed");
}
```

`final`

The `final` keyword is used to indicate that something cannot be changed further: - `final` variable → once assigned, its value **cannot be reassigned** (essentially a constant). - `final` method → **cannot be overridden** by any subclass, useful when you want to lock down specific behavior. - `final` class → **cannot be inherited/extended** at all (e.g., Java's built-in `String` class is `final`, which is part of why Strings are safe to share and cache).

`this`

`this` is a reference to the **current object** — the object whose method or constructor is currently executing. It's most commonly used to distinguish between an instance variable and a parameter/local variable that share the same name:

```
class Student {
    int age;
    Student(int age) {
        this.age = age;    // this.age refers to the instance field; age (without this) refers to the constructor parameter
    }
}
```

11. Packages

A package is essentially a namespace — a way of grouping related classes and interfaces together. Packages help avoid naming conflicts (two classes with the same name can coexist if they're in different packages), make large codebases more organized, and allow access control at the package level (via default/protected modifiers).

```
package com.myapp.utils;    // declares which package the current file's classes belong to
import java.util.ArrayList; // imports one specific class so it can be used without a full qualified name
import java.util.*;         // imports every class from a package (convenient, though sometimes discouraged for clarity)
```

- `java.lang` is imported automatically into every Java file (which is why you can use `String`, `Math`, `System`, etc. without an explicit import).
- Other commonly used built-in packages include `java.util` (collections, utilities), `java.io` (input/output), `java.net` (networking), and `java.sql` (database access).

12. Exception Handling

An exception is an event that disrupts the normal flow of a program's instructions, typically caused by an error condition (like dividing by zero, or accessing an invalid array index). Java's exception-handling mechanism lets a program detect and respond to these problems gracefully, rather than crashing outright.

Exception Hierarchy (explained): All exception-related classes descend from `Throwable`. It splits into two branches — `Error`, representing serious problems the application generally shouldn't try to catch (like running out of memory), and `Exception`, representing conditions a program might reasonably want to catch and handle.

```
Throwable
|-- Error (serious, unrecoverable -- e.g., OutOfMemoryError)
`-- Exception
    |-- Checked Exceptions (checked at compile time -- must be handled or declared, e.g., IOException)
    `-- Unchecked Exceptions / RuntimeException (not checked at compile time, usually programming bugs, e.g.,
        NullPointerException, ArithmeticException)
```

```
try {
    int result = 10 / 0; // this line throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage()); // handles the exception gracefully
} finally {
    System.out.println("This always executes"); // runs regardless of whether an exception occurred
}
```

Keyword-by-keyword explanation: - `try` — wraps the block of code that might throw an exception; the JVM monitors this block for problems. - `catch` — catches and handles a specific type of exception if it's thrown inside the corresponding `try` block; you can have multiple `catch` blocks for different exception types. - `finally` — a block that **always executes**, whether or not an exception was thrown or caught, typically used for cleanup work like closing files or database connections. - `throw` — used inside your code to manually/explicitly trigger (throw) an exception object. - `throws` — used in a method's signature to declare that the method might throw a particular checked exception, so callers know they need to handle it.

```
void checkAge(int age) throws Exception {
    if (age < 18) throw new Exception("Not eligible"); // manually throwing an exception with a custom message
}
```

Custom Exception — you can create your own exception types by extending `Exception` (checked) or `RuntimeException` (unchecked), useful for representing domain-specific error conditions clearly:

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) { super(msg); } // passes the message up to the Exception class
}
```

- **Multi-catch** — lets one `catch` block handle several exception types together, reducing duplicate code: `catch (IOException | SQLException e) { }`
- **try-with-resources** — automatically closes resources (like files or streams) once the try block finishes, even if an exception occurs, so you don't have to manually call `close()`:

```
try (FileReader fr = new FileReader("file.txt")) {
    // use fr -- it will be automatically closed at the end of this block
}
```

13. String Handling

A `String` in Java represents a sequence of characters, and importantly, Strings are **immutable** — once a `String` object is created, its content can never be changed. Any operation that appears to “modify” a `String` (like concatenation) actually creates and returns a brand-new `String` object instead.

```
String s1 = "Hello";           // stored in a special memory area called the "String Pool" (for reuse/efficiency)
String s2 = new String("Hello"); // explicitly forces creation of a new object on the Heap, bypassing the pool

s1 == s2;                      // false -- compares references (memory addresses), which are different here
s1.equals(s2);                 // true  -- compares actual character content, which is the same
```

This distinction between `==` and `.equals()` is one of the most common sources of bugs for beginners: always use `.equals()` to compare the actual text content of Strings.

Common methods: `length()` (number of characters), `charAt(i)` (character at index *i*), `substring(start, end)` (extracts part of the string), `indexOf()` (position of a substring), `toUpperCase()` / `toLowerCase()`, `trim()` (removes leading/trailing whitespace), `replace()`, `split()` (breaks into an array based on a delimiter), `concat()`, `equalsIgnoreCase()`.

StringBuilder / StringBuffer

Because Strings are immutable, repeatedly modifying a `String` in a loop (e.g., concatenating inside a loop) is inefficient, since it creates a new object every single time. `StringBuilder` and `StringBuffer` solve this by providing **mutable** character sequences that can be changed in place without creating new objects each time.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World"); // modifies the SAME object in memory, rather than creating a new one
```

- `StringBuilder` — faster, but **not** thread-safe (should not be shared across threads without external synchronization).
- `StringBuffer` — thread-safe because its methods are synchronized, but this synchronization makes it somewhat slower.

14. Wrapper Classes

Java’s Collections Framework and many APIs are designed to work only with **objects**, not primitive values. Wrapper classes exist to “wrap” a primitive value inside an object, so primitives can be used wherever objects are required (e.g., stored in an `ArrayList<Integer>`, which cannot directly hold `int` values).

Primitive	Wrapper
<code>int</code>	<code>Integer</code>
<code>char</code>	<code>Character</code>
<code>double</code>	<code>Double</code>
<code>boolean</code>	<code>Boolean</code>

```
int a = 5;
Integer obj = a; // Autoboxing -- Java automatically converts the primitive into its wrapper object
int b = obj;     // Unboxing -- Java automatically converts the wrapper object back into a primitive
```

This automatic conversion (introduced in Java 5) means you rarely have to manually wrap/unwrap values yourself, though it’s important to understand it’s happening, since it can have small performance implications in tight loops.

15. Collections Framework

Arrays have a fixed size and limited built-in functionality. The Collections Framework provides a much richer, unified set of classes and interfaces for storing, retrieving, and manipulating groups of objects dynamically — growing and shrinking as needed, and offering ready-made algorithms for searching, sorting, etc.

```
Collection (interface)
|-- List (ordered, allows duplicates) -> ArrayList, LinkedList, Vector
|-- Set (no duplicates) -> HashSet, LinkedHashSet, TreeSet
`-- Queue (FIFO) -> PriorityQueue, ArrayDeque
```

```
Map (interface, key-value pairs, technically NOT a Collection) -> HashMap, LinkedHashMap, TreeMap, Hashtable
```

- **List** — maintains insertion order and allows duplicate elements; good when order matters or when you need indexed access.

- **Set** — does not allow duplicate elements; good when uniqueness is the priority.
- **Queue** — designed for holding elements prior to processing, typically in First-In-First-Out order.
- **Map** — stores data as key-value pairs, where each key is unique and maps to exactly one value; extremely useful for fast lookups.

Examples:

```
List<String> list = new ArrayList<>();
list.add("A"); list.add("B");           // maintains order: [A, B]

Set<Integer> set = new HashSet<>();
set.add(1); set.add(1);                 // second add is ignored -- duplicates aren't allowed

Map<String, Integer> map = new HashMap<>();
map.put("apple", 10);
map.get("apple");                       // retrieves 10 using the key "apple"
```

Class	Ordering	Duplicates	Thread-Safe
ArrayList	insertion order	yes	no
LinkedList	insertion order	yes	no
HashSet	no guaranteed order	no	no
TreeSet	sorted order	no	no
HashMap	no guaranteed order	keys unique	no
TreeMap	sorted by key	keys unique	no

- **Iterator** — a standard object used to step through the elements of any collection one at a time, regardless of the underlying collection type, using `hasNext()` (checks if more elements exist), `next()` (returns the next element), and `remove()` (safely removes the current element during iteration).

16. File Handling (I/O)

Java's I/O (Input/Output) classes let a program read data from, and write data to, external sources like files, keeping the data even after the program finishes running (unlike variables in memory, which are lost).

```
import java.io.*;

// Writing to a file
FileWriter fw = new FileWriter("file.txt");
fw.write("Hello File");
fw.close();           // always close streams to release system resources and ensure data is flushed to disk

// Reading from a file
FileReader fr = new FileReader("file.txt");
BufferedReader br = new BufferedReader(fr); // wraps FileReader to read efficiently, line by line
String line;
while ((line = br.readLine()) != null) {    // readLine() returns null once the end of the file is reached
    System.out.println(line);
}
br.close();
```

- The `File` class itself represents a file or directory **path** on the filesystem — it doesn't read/write content directly, but is used for things like checking if a file exists, creating/deleting it, or listing directory contents.
- Streams come in two flavors: **Byte streams** (`InputStream` / `OutputStream` and their subclasses) handle raw binary data (e.g., images), while **Character streams** (`Reader` / `Writer` and their subclasses) are optimized for handling text data with proper character encoding.

17. Memory Management

Understanding how Java manages memory helps explain the behavior of objects, variables, and garbage collection.

- **Heap** — the memory area where all objects (and their instance variables) are actually stored. It is shared across the entire application and managed by the garbage collector.
- **Stack** — stores method call frames, local variables, and partial results. Each thread has its own stack. Memory here is automatically reclaimed as soon as a method finishes (Last-In-First-Out order).
- **Garbage Collection (GC)** — Java automatically identifies objects on the heap that are no longer reachable/referenced by any part of the running program, and reclaims their memory, so developers don't need to manually free memory (unlike C/C++).

- `System.gc()` — merely **requests** that the JVM run garbage collection; it is only a suggestion and is not guaranteed to run immediately, or at all, right away.
- `finalize()` — a method that used to be called just before an object was garbage collected, giving it a chance to clean up; it has been deprecated since Java 9 in favor of more predictable mechanisms like try-with-resources.

Quick Revision — Key Terms

Term	Meaning
JIT Compiler	Just-In-Time compiler; converts frequently executed bytecode into native machine code at runtime for better performance
Ternary operator	<code>?:</code> — a shorthand way of writing a simple if-else that produces a value
Overloading	Same method name, different parameter lists, resolved at compile-time
Overriding	Subclass redefines a parent method with the identical signature, resolved at runtime
Upcasting	Referring to a subclass object using a reference of its superclass type
Immutable	An object whose internal state can never change after it's created (e.g., String)
Marker interface	An interface with no methods at all, used just to “tag” a class with some special meaning (e.g., <code>Serializable</code> , <code>Cloneable</code>)

These notes cover Core Java fundamentals commonly required for interviews, exams, and foundational application development.